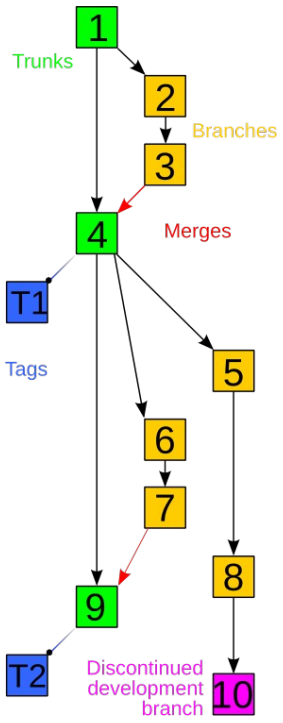
Agenda – Sessão 2

1) Recap
   - VCS (Git)
   - Python
   - (3.X please)
   - Create personal git remote account (eg: GitHub, Bitbucket or any other)
   - IDE
2) Today
   - Basic git commands
   - Markdown (Documentação)
   - Python
     - Intro
     - (“Não vamos falar de algoritmia nem de o que é programar”)
     - Comentários / Documentação
     - Variáveis

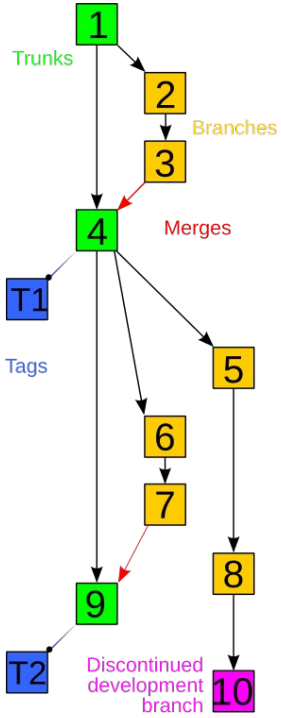
Basic git commands

- Git status
- Git clone (ter uma cópia em local de um repositório remoto)
  - git clone https://github.com/MontesDaLua/ollama
- Git pull receber as atualizações de um repositório remoto
  - git pull https://github.com/MontesDaLua/ollama **master**
- Git remote (ver os remotos)
  - git remote -v
- Git branch (strategy trunk based)
  - Criar
    - Git branch oMeuNovoBranch
  - Ver
    - Git branch
  - Apagar
    - Git branch -d oMeuNovoBranch



Basic git commands

- Git log
- Checkout
  - Git checkout umBranchExistente
- Git commit (criar um snapshot único do “texto”)
  - Git commit -a -m “ O meu novo maravilhos commit que faz qualquer coisa “
- Git merge (juntar 2 branch)
  - Git merge umOutroBranchExistente
- Git push (enviar para o repositório remoto as modificações)



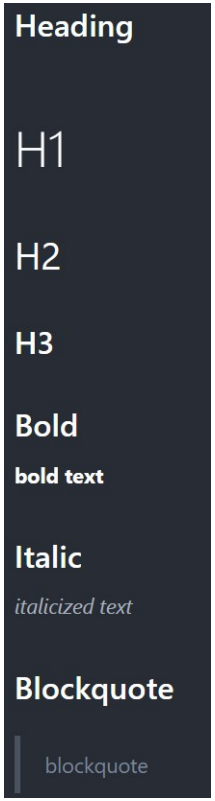
Markdown [1]

- Escrever documentação (até de código fonte)
- Simples
- Pode ser convertido em outros formatos (pdf, “Word”, (X)HTML)
- Usada por vários “site (eg:GitHub)

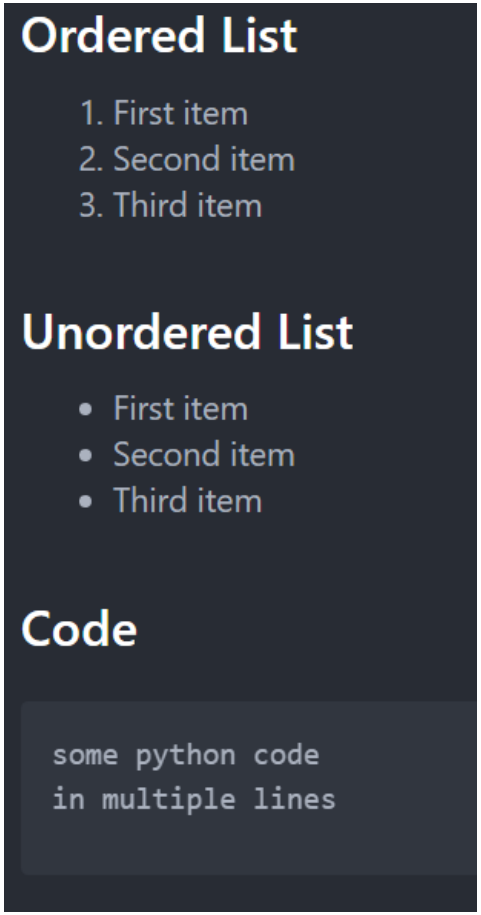
1 - https://en.wikipedia.org/wiki/Markdown

Markdown Sintaxe -1

Heading
H1
H2
H3
Bold
bold text
Italic
italicized text
Blockquote
> blockquote



Ordered List
1. First item
1. Second item
1. Third item
Unordered List
- First item
- Second item
- Third item
Code
```
some python code
in multiple lines
```



1 - https://www.markdownguide.org

Markdown Sintaxe -2

Horizontal Rule

Link
[Markdown Guide](https://www.markdownguide.org)

Image

![alt text](https://www.markdownguide.org/assets/images/tux.png)

Horizontal Rule

Link

Markdown Guide

Image

Table
Syntax	Description
Header	Title
Paragraph	Text

Strikethrough
~~The world is flat.~~
Task List
- [x] Write the press release
- [ ] Update the website
- [ ] Contact the media

Table

Syntax	Description
Header	Title
Paragraph	Text

Strikethrough

~~The world is flat.~~

Task List

- [x] Write the press release
- [ ] Update the website
- [ ] Contact the media

Python

- Python is a high-level, general-purpose programming language. Its design philosophy emphasizes code readability with the use of significant indentation.
- Python is dynamically typed and garbage-collected.
- Supports multiple programming paradigms
  - Structured
  - Object-oriented
  - Functional programming.
- Have a comprehensive standard library(s) & Ecosystem

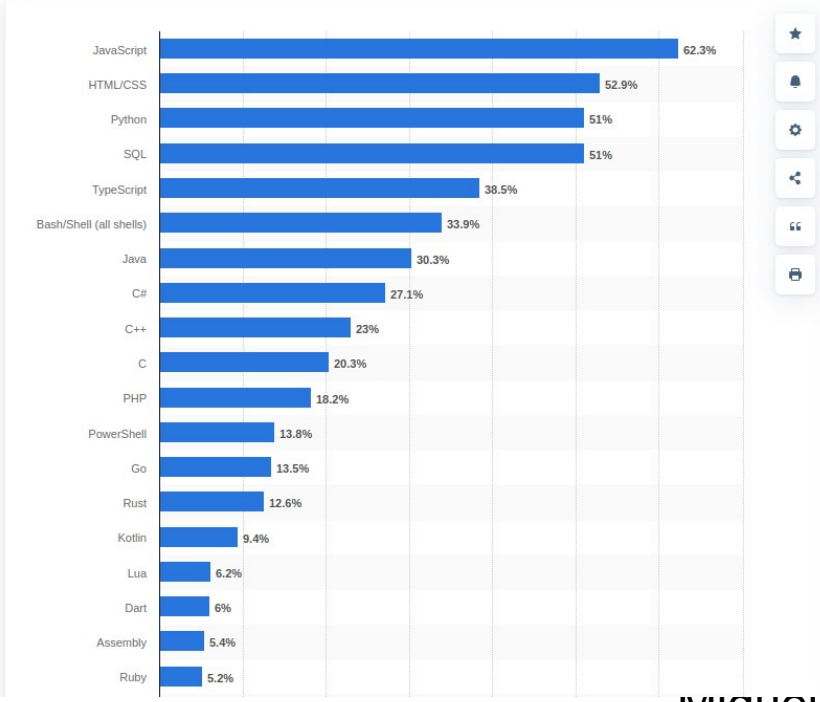
Python - Zen

- Beautiful is better than ugly.
- Explicit is better than implicit.
- Simple is better than complex.
- Complex is better than complicated.
- Readability counts!

Python - Stats

Technology & Telecommunications › Software

Most used programming languages among developers worldwide as of 2024



DOWNLOAD

PDF + XLS + PNG + PPT +

Source
- Show sources information
- Show publisher information
- Use Ask Statista Research Service

Release date
July 2024

Region
Worldwide

Survey time period
May 19 to June 20, 2024

Number of respondents
60,171 respondents

Special properties
Software developers

Method of interview
Online survey

Supplementary notes

Original question: "Which programming,

https://www.statista.com/statistics/793628/worldwide-developer-survey-most-used-languages/

Comentários – “A” instrução mais importante em Python

```
# Este é um comentário de linha única
# Isto são Docstrings
'''
Este é um comentário de  múltiplas linhas.
Pode ocupar várias linhas de texto.
'''
"""
Este também é um comentário de múltiplas linhas,
usando aspas duplas.
"""
```

Variáveis (Qual a diferença para uma constante ?)

- Tipo de variável é implicito
- Tudo é python é **Case Sensitive**
  - **X != x**
  - **PI = 3.141592657**
  - **Pi = 3.141592657**
- **Estilos**
  - Variáveis (Vamos falar de estilos e regras mais tarde)
  - **Camel Case**
    - Cada palavra em um nome composto começa com uma letra maiúscula, exceto a primeira palavra (**minhaVariavel**)
  - **Pascal Case**
    - Cada palavra em um nome composto começa com uma letra maiúscula (**MinhaVariavel**)
  - **Snake case**
    - Cada palavra separada por um underscore (**minha_variavel**)
- Pylint – Static code analyser - (como instalar no final da sessão)
  - Examina o código Python, não executa.
  - Encontrar erros
  - Inconsistências de estilo
  - Código mal escrito
  - Potenciais problemas de qualidade.

Tipos de Dados em Python

1) **Integers** (int): Números inteiros, como 1, 2, 3, etc. Exemplo: x = 5
2) **Floats** (float): Números decimais, como 3.14 ou -0.5. Exemplo: x = 3.14
3) **Strings** (str): Sequências de caracteres, como "hello" ou 'hello'. Exemplo: x = "hello"
4) **Boolean** (bool): Valores lógicos, como True ou False. Exemplo: x = True
5) **Listas** (list): Coleções de valores, como [1, 2, 3] ou ["a", "b", "c"]. Exemplo: x = [1, 2, 3]
6) **Tuplas** (tuple): Coleções imutáveis de valores, como (1, 2, 3) ou ("a", "b", "c"). Exemplo: x = (1, 2, 3)
7) **Dicionários** (dict): Coleções de pares chave-valor, como {"nome": "João", "idade": 30}. Exemplo: x = {"nome": "João", "idade": 30}
8) **None**: Valor **especial** que representa a ausência de valor. Exemplo: x = None
9) **Complex** (complex): Números complexos, como 3 + 4j. Exemplo: x = 3 + 4j
10) **Bytes**
    1) Bytes (bytes): Sequências de bytes, como b'hello'. Exemplo: x = b'hello'
    2) Bytes-like (bytearray): Sequências de bytes mutáveis, como bytearray(b'hello'). Exemplo: x = bytearray(b'hello')
11) **Conjuntos**
    1) Set (set): Coleções não ordenadas de valores únicos, como {1, 2, 3} ou {"a", "b", "c"}. Exemplo: x = {1, 2, 3}
    2) Frozen set (frozenset): Coleções não ordenadas e imutáveis de valores únicos, como frozenset({1, 2, 3}) ou frozenset({"a", "b", "c"}). Exemplo: x = frozenset({1, 2, 3})

Numéricos

```
# Inteiros (int)
int1 = 10
print(int1)

# Floats (float)
float1 = 3.14
print(float1)


# Complexos
comp1 = 2 + 3j
print(comp1)


# Operaçõs aritméticas
soma = int1 + float1
dif = int1 - float1
multiplicacao = int1 * float1
divisao = int1 / float1
modulo = float1 % 2
potencia = int1 ** 2


print(f"Soma: {soma}")
print(f"Diferenca: {dif}")
print(f"Multiplicacão: {multiplicacao}")
print(f"Divisão: {divisao}")
print(f"Módulo: {modulo}")
print(f"Potencia: {potencia}")
```

```
10
3.14
(2+3j)
Soma: 13.14
Diferenca: 6.859999999999999
Multiplicaçãon: 31.400000000000002
Divisião: 3.184713375796178
Módulo: 1.1400000000000001
Potencia: 100
```

- Função “print”
  - Sem Formatação
  - Com Formatação (fast strings “à lá printf”)

Booleanos (e condicionais)

```
# Variáveis booleanas
verdadeiro = True
falso = False

# Operações booleanas
print(f"Verdadeiro AND Falso: {verdadeiro and
    falso}")
print(f"Verdadeiro OR Falso: {verdadeiro or falso}")
print(f"NOT Verdadeiro: {not verdadeiro}")
```

```
Verdadeiro AND Falso: False
Verdadeiro OR Falso: True
NOT Verdadeiro: False
```

- Álgebra de Boole - **Qualquer condição pode ser expressa com a combinação de**
  - and
  - or
  - not

Condições Básicas

```
# Comparação
a = 10
b = 5

print(f"A é igual a B: {a == b}")
print(f"A é diferente de B: {a != b}")
print(f"A é maior que B: {a > b}")
print(f"A é menor que B: {a < b}")
print(f"A é maior ou igual a B: {a >= b}")
print(f"A é menor ou igual a B: {a <= b}")
```

```
A é igual a B: False
A é diferente de B: True
A é maior que B: True
A é menor que B: False
A é maior ou igual a B: True
A é menor ou igual a B: False
```

- Igualdade
- Desigualdade
- Maior/Menor

Strings

```
# Strings são sequências de caracteres, usadas
    #para representar texto.

string1 = "Viva a todos!"
string2 = 'Python é uma linguagem de prgramação!'
string3 = """Esta é uma string
com múltiplas linhas."""

print(string1)
print(string2)
print(string3)
```

```
Viva a todos!
Python é uma linguagem de prgramação!
Esta é uma string
com múltiplas linhas.
```

- strings são sequências de caracteres, como letras, números e símbolos, que são encerradas entre aspas simples (') , duplas ou triplas.
- São usadas para representar texto sendo um dos tipos de dados mais comuns.

Strings – Métodos Simples

```
# 1. Concatenação
nome = "Miguel"
apelido = "Rodrigues"
nome_completo = nome + " " + apelido
print(nome_completo)
# 2. Tamanho da string
tamanho = len(nome_completo)
print(tamanho)
# 3. Maiúsculas e minúsculas
texto = "Python é uma linguagem de programação."
texto_maiusculo = texto.upper()
print(texto_maiusculo)
texto_minusculo = texto.lower()
print(texto_minusculo)
# 4.Substituição
novo_texto = texto.replace("Python", "Java")
print(novo_texto)
# 5. Split(dividir)
frase = "Esta é uma frase com várias palavras."
palavras = frase.split()
print(palavras)
# 6. Verificação de inicio e fim
frase = "Olá, mundo!"
print(frase.startswith("Olá"))
print(frase.endswith("mundo!"))
```

```
Miguel Rodrigues
16
PYTHON É UMA LINGUAGEM DE PROGRAMAÇÃO.
python é uma linguagem de programação.
Java é uma linguagem de programação.
['Esta', 'é', 'uma', 'frase', 'com', 'várias', 'palavras.']
True
True
```

- Manipulação
- Atributos
- Transformação em listas (falamos de listas mais tarde)

Strings - Slicing – Também aplicado a listas

```
# String slicing
string = "Hello, world!"
# Accessing individual characters. Index
    always start with 0
first_char = string[0]
last_char = string[-1]
# Slicing a substring
substring = string[0:5]  # From index 0 (inclusive) to
    5 (exclusive)
another_substring = string[7:]  # From index 7 to
    the end
yet_another_substring = string[:5]  # From the
    beginning to index 5 (exclusive)
print(f"First character: {first_char}")
print(f"Last character: {last_char}")
print(f"Substring: {substring}")
print(f"Another substring: {another_substring}")
print(f"Yet another substring:
    {yet_another_substring}")
# Reverse a string
reversed_string = string[::-1]
print(f"Reversed string: {reversed_string}")
# Extract every second character
every_second_char = string[::2]
print(f"Every second character:
    {every_second_char}")
```

```
First character: H
Last character: !
Substring: Hello
Another substring: world!
Yet another substring: Hello
Reversed string: !dlrow ,olleH
Every second character: Hlo ol!
```

- Index começa sempre em zero
- Strings podem ser tratadas como listas de caracteres
- Index podem ser negativos (começar no fim)

Função Input (mais tarde falaremos do que é uma função)

```
nome = input("Qual é o seu nome? ")
print("Olá, " + nome + "!")
```

```
Qual é o seu nome? Miguel
Olá, Miguel!
```

- A função input() em Python é usada para obter entrada do utilizador.
- É uma função built-in que permite obter uma entrada de **strings**.

Condições - If , elif and Else

```
idade = 25
vencimento = 2000

if idade >= 18 and vencimento >= 1500:
    print("Pode pedir um empréstimo.")
elif idade >= 18:
    print("É maior de idade, mas o  vencimento não é
     suficiente.")
else:
    print("Menor de idade. Esqueça Empréstimos
     Bancários")
```

```
Pode pedir um empréstimo.
```

- Condição Booleana inicial
- Condição Booleana subsequente
- “e se nada for satisfeito ” **else**

“Descobrir” os Tipos de Dados

```
a = 5
b = 3.14
c = True
d = [1, 2, 3]
e = (1, 2, 3)
f = {"nome": "João", "idade": 30}
g = None
h = 3 + 4j
I = {1, 2, 3}
j = frozenset({1, 2, 3})
print(type(a))
print(type(b))
print(type(c))
print(type(d))
print(type(e))
print(type(f))
print(type(g))
print(type(h))
print(type(I))
print(type(j))
```

- <class 'int'>
- <class 'float'>
- <class 'bool'>
- <class 'list'>
- <class 'tuple'>
- <class 'dict'>
- <class 'NoneType'>
- <class 'complex'>
- <class 'set'>
- <class 'frozenset'>

- O que retorna a função **type ?**
- **Qual a** != entre “i” e “I”
  - frozenset

Casting - casting é utilizado para converter um valor de um tipo de dado para outro.

```
a = int("123")
b = float("3.14")
c = str(123)
d = bool(1)
e = list("hello")
f = tuple("hello")
g = dict(name="João", age=30)
h = complex(3, 4)
i = bytes("hello", "utf-8")
j = bytearray(b"hello")
k = set([1, 2, 3])
l = frozenset([1, 2, 3])
```

- a é agora 123
- b é agora 3.14
- c é agora "123"
- d é agora True
- e é agora ["h", "e", "l", "l", "o"]
- f é agora ("h", "e", "l", "l", "o")
- g é agora {"name": "João", "age": 30}
- h é agora (3+4j)
- i é agora b'hello'
- j é agora bytearray(b'hello')
- k é agora {1, 2, 3}
- l é agora frozenset({1, 2, 3}

Excepções

```
3/0

try:
  3/0
except ZeroDivisionError:
  print ("Divisão por zero é
    impossível para um numérico")


try:
  "abc"/0
except ZeroDivisionError:
  print ("Divisão por zero é impossível para
    um numérico")
except:
  print("Não sei como lidar com a divisão
    de uma string por zero ")
```

- ZeroDivisionError: division by zero
- Divisão por zero é impossível para um numérico

- Sempre testar as condições adversas
- Especificar quais as condições “adversas”
- Que podem ser encadeadas

Excepções – Porque usar (Plan for the worst , Hope for the best)

1) **Melhora o código**: Ao tratar as excepções, seu programa/script fica mais resistente aos erros "inesperados".
2) **Facilita "debug"**: As mensagens de erro fornecidas pelas exceções ajudam a identificar a causa do problema.
3) **Permite um fluxo de controle mais flexível**: Podemos personalizar o comportamento do script/programa em caso de erros.

Tipos de Dados - Listas -1

```
lista1 = [1, 2, 3, 1.0 , "Olá", "Mundo"]
# (1)
type(lista1[4])


#2
type(lista1[4:5]) # start/end/step

#3
Lista1[4:5][0]

El1 = lista1[0]
el1  = lista1[-1]

#4
print(lista1)
lista1[3] = "ABC"
print(lista1)

$5
lista1.append("NoFinall")
print(lista1)

# 6
lista1.remove("Olá")
print(lista1)
```

```
1) Str
2) list
3) ”Olá”
4)
    [1, 2, 3, 1.0, 'Olá', 'Mundo']
    [1, 2, 3, 'ABC', 'Olá', 'Mundo']
5)  [1, 2, 3, 'ABC', 'Olá', 'Mundo', 'NoFinall', 'NoFinall']
6) [1, 2, 3, 'ABC', 'Mundo', 'NoFinall', 'NoFinall']
```

- Listas são uma estrutura de dados que permite armazenar uma coleção de elementos (de qualquer tipo), ordenados e mutáveis.
- Podemos aceder aos elementos da lista com os mesmos métodos que vimos para strings.
- Podemos “encadear”
- Significa que podemos modificar o conteúdo da lista após sua criação **(Métodos)** .
  - Alterar
  - Adicionar
  - Remover

Tipos de Dados - Listas -2

```
# Tamanho da Lista (A)
print(len(lista1))

# Verificar Existência (B)
print("34" in lista1)

# shortcut's
# pop  (C)

print(lista1.pop())
print(lista1)

# ordenar  (D)
x = [1,3,-1]
Y = x.sort()
print(y)
```

A) 7
B) False
C)
NoFinall
[1, 2, 3, 'ABC', 'Mundo']

D)[-1, 1, 3]

- Listas outros **Métodos** (mais comuns)
  - **len**
  - **Existência (in)**
  - **Outras funções**
    - **pop**

Tipos de Dados - Dicionários

```
# Definir (A)
dict1 = {
   "nome": "Alice",
   "idade": 30,
   "cidade": "Lisboa"
   }

print(dict1)

# Adicionar chave (B)
dict1["OutraChave"] = 135.23
print(dict1)

# Ver Valor (C)
print(dict1["cidade"])

# Remover uma chave (D)
del dict1["nome"]
print(dict1)

# Chaves Existentes (E)
dict1.keys()
```

A) {'nome': 'Alice', 'idade': 30, 'cidade': 'Lisboa'}
B) {'nome': 'Alice', 'idade': 30, 'cidade': 'Lisboa', 'OutraChave': 135.23}
C) Lisboa
D) {'idade': 30, 'cidade': 'Lisboa', 'OutraChave': 135.23}
E) dict_keys(['idade', 'cidade', 'OutraChave'])

- Definir
- Conceito “Chave”(Key)

Chave(Key)/Unicidade/ Dados Compostos

- Somos “únicos” ou “temos várias chaves ” ? https://www.youtube.com/watch?v=W0YfQgkskHM (minuto 11 s.f.f. ;-))

Actividade da Sessão

- Como definir
  - Uma lista de listas ?
  - Uma dicionário de listas ?
  - Uma lista de dicionários
  - Instalar no seu ambiente local
    - Pylint